从 .raw 到 RGB

按三域流程手写 ISP Pipeline

20 个 stage · 5760×3840 · 约 13 秒

BLC → Demosaic → CCM → Gamma → YUV → PNG



输入 .raw + .meta · 输出 RGB PNG

5 分钟讲解 · 2026

为什么 .raw 不是一张图片?

- 只有像素，没有宽高 / CFA / 黑电平
- 传感器是 Bayer 阵列：每个像素只有 R/G/B 之一
- 旁边必须有 .meta ， ISP 才能解释它

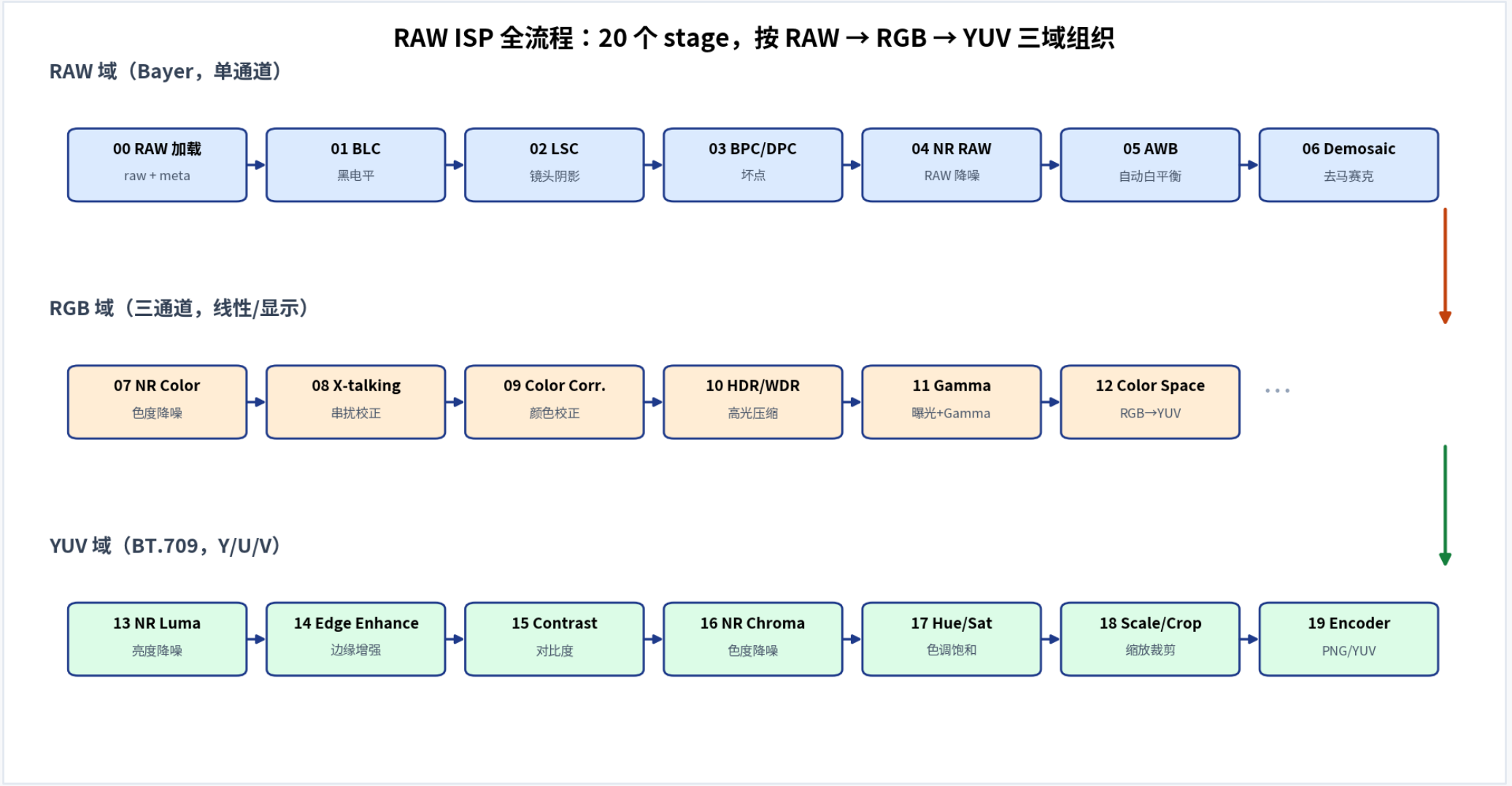
ISP = 把 Bayer 单通道
变成可显示 RGB 的 20 步



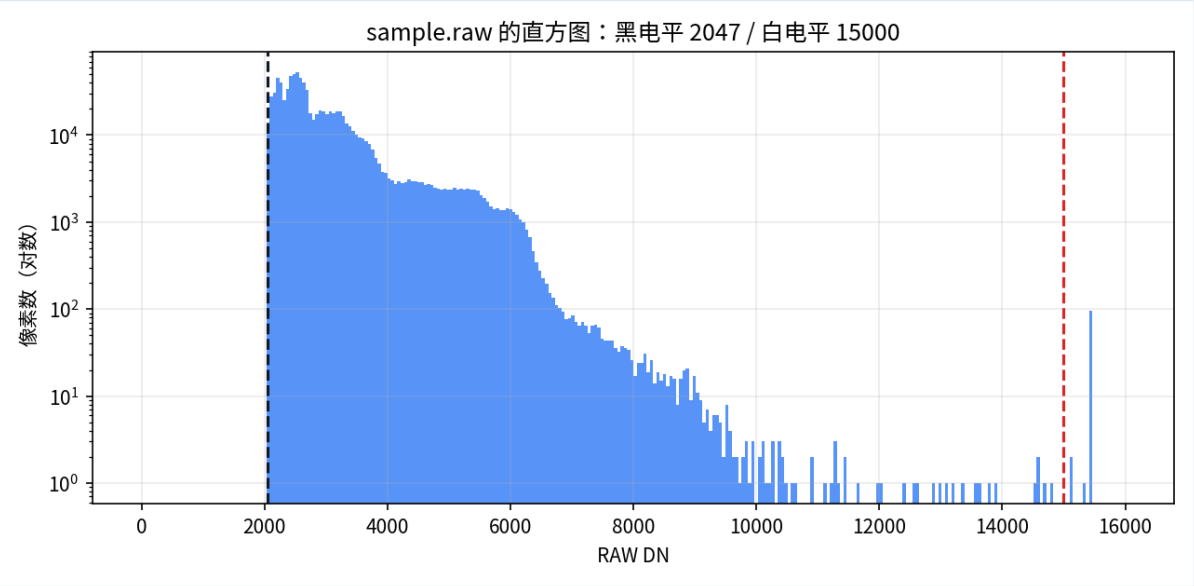
RGGB： G 占 50%， R/B 各占 25%

data/sample.raw + data/sample.meta

20 个 stage，三个域



先修传感器和镜头



黑电平 2047 · 白电平 15000 · 16 bit

先减黑电平，再乘增益。
否则暗部会被放大成偏色。

01 **BLC**
黑电平

02 **LSC**
镜头阴影

03 **BPC/DPC**
坏点

04 **NR RAW**
RAW 降噪

05 **AWB**
自动白平衡

06 **Demosaic**
颜色插值

每个像素只有一个颜色，怎么变成 RGB？

去马赛克：100% 裁切

双线性



Malvar-He-Cutler



100% 裁切：左双线性，右 MHC

G = (G 左 +G 右 +G 上 +G 下)/4
MHC： 5×5 卷积 + 梯度修正

双线性

邻居平均，快；
边缘拉链纹、伪彩。

Malvar-He-Cutler

梯度修正，伪彩更少；
本课默认。

颜色、动态范围、 Gamma

Stage 09 前后对比（同一张 .raw，同一次流水线）

上一阶段：08 X-talking



本阶段：09 Color Correction



CCM 前（左） vs CCM 后（右）

07 **NR Color**
色度降噪

08 **X-talking**
串扰校正

09 **Color Corr.**
相机→ sRGB

10 **HDR/WDR**
高光压缩

11 **Gamma**
曝光 +sRGB

12 **Color Space**
RGB→YUV

最后修观感，再编码

Stage 12 前后对比（同一张 .raw，同一次流水线）

上一阶段：11 Gamma



本阶段：12 Color Space



RGB→YUV 之后： Y/U/V 三通道

13 **NR Luma**
亮度降噪

14 **Edge Enhance**
边缘增强

15 **Contrast**
对比度

16 **NR Chroma**
色度降噪

17 **Hue/Sat**
色相饱和

18/19 **Scale/Encoder**
缩放 / 编码

为什么这套代码适合学?

```
stages/06_demosaic/
  README.md     theory
  stage.cpp     algorithm
  main.cpp      CLI

common/
  raw.cpp       .raw + .meta
  image.h       .isp container
  color.h       sRGB / matrix

data/
  sample.raw
  sample.meta
```

一个目录 = 一节实验课

一个 stage 一个函数

接口在 stages.h ，改完再编译。

中间格式可检查

.isp 带头信息， isp_dump 随时看。

两种跑法

内存流水线或 --steps 分步。

改算法 → 再编译 → 13 秒看到结果

跑通： 2880×1620 ， 约 13 秒



out/final.png ・ sRGB PNG

20 个 stage

13 秒 / 全流程

0 分步 vs 内存差异

2.9K ×1.6K 输出

同一份 .raw ， 两种输出

把三域拆开，ISP 就不再是黑魔法

01 **RAW 域修数据**

BLC 、 LSC 、 DPC 、 NR 、 AWB 、 Demosaic

02 **RGB 域修颜色**

NR Color 、 X-talking 、 CCM 、 WDR 、 Gamma

03 **YUV 域修观感**

亮度 / 色度降噪、边缘、对比度、 Hue/Sat

先跑通 → 再拆开 → 最后替换算法

谢谢

欢迎提问 / 一起改代码

`docs/PRINCIPLES.md`